

Airflow - SubDags, Sensors and Xcom

SubDags:

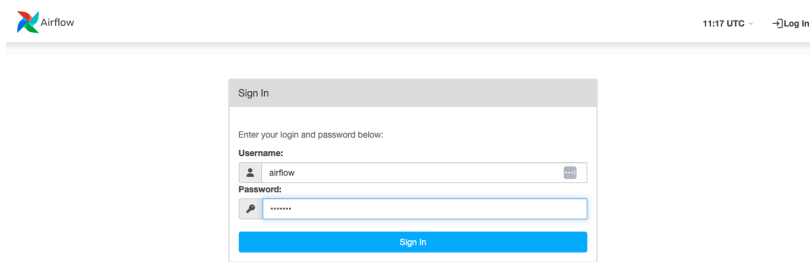
Create my_subdag.py file under dags/

```
my_subdag.py U X
tascientest-data-engineering > airflow > dags > my_subdag.py
1 from airflow import DAG
2 from airflow.operators.bash import BashOperator
3 from airflow.utils.dates import days_ago
4
5
6 def create_sub_dag(dag_id, schedule_interval, start_date):
7     my_sub_dag = DAG(
8         dag_id=dag_id,
9         schedule_interval=schedule_interval,
10        tags=['tutorial', 'datascientest'],
11        default_args={
12            'start_date': days_ago(0)
13        }
14    )
15
16    task1 = BashOperator(
17        bash_command="echo subdag task 1",
18        task_id="my_sub_dag_task1",
19        dag=my_sub_dag
20    )
21
22    task2 = BashOperator(
23        bash_command="echo subdag task 2",
24        task_id="my_sub_dag_task2",
25        dag=my_sub_dag
26    )
27
28    task1 >> task2
29
30    return my_sub_dag
```

Create parent_dag.py file under dags/

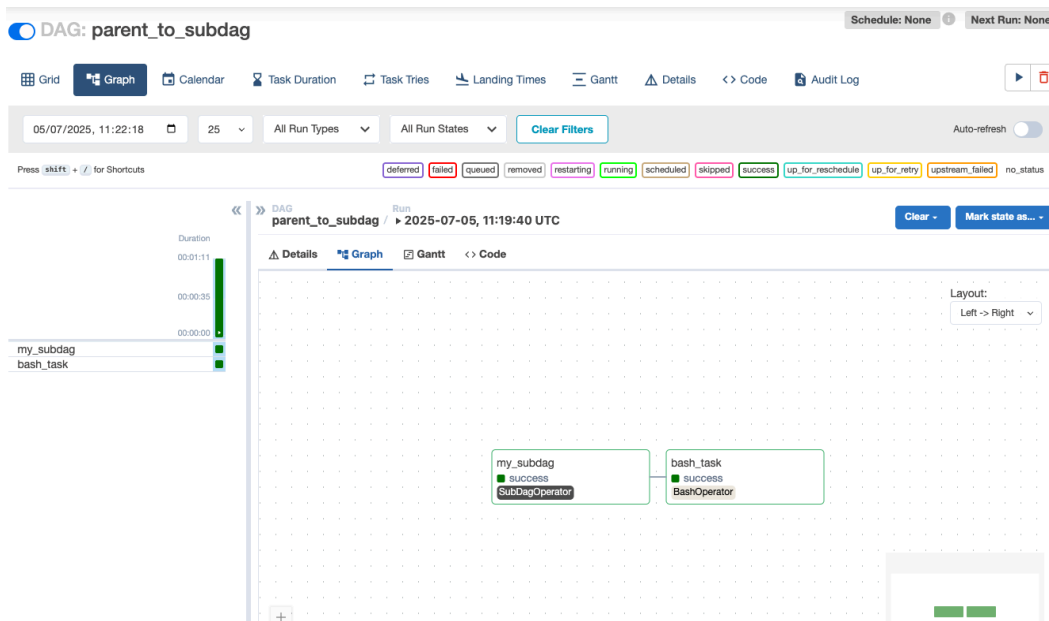
```
parent_dag.py U X
tascientest-data-engineering > airflow > dags > parent_dag.py
1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.operators.subdag import SubDagOperator
4 from airflow.operators.bash import BashOperator
5 from my_subdag import create_sub_dag
6
7
8 my_parent_dag = DAG(
9     dag_id="parent_to_subdag",
10    schedule_interval=None,
11    tags=['tutorial', 'datascientest'],
12    default_args={
13        'owner': 'airflow',
14        'start_date': days_ago(0, 1)
15    }
16 )
17
18 task1 = SubDagOperator(
19     task_id="my_subdag",
20     subdag=create_sub_dag(
21         dag_id=my_parent_dag.dag_id + '.' + 'my_subdag',
22         schedule_interval=my_parent_dag.schedule_interval,
23         start_date=my_parent_dag.start_date,
24         dag=my_parent_dag
25     )
26 )
27
28 task2 = BashOperator(
29     task_id="bash_task",
30     bash_command="echo hello world from parent",
31     dag=my_parent_dag
32 )
33
34 task1 >> task2
```

Open URL <http://18.203.126.172:8080/> (18.203.126.172 is the IP of the VM, at port 8080). Enter the username airflow and password airflow.

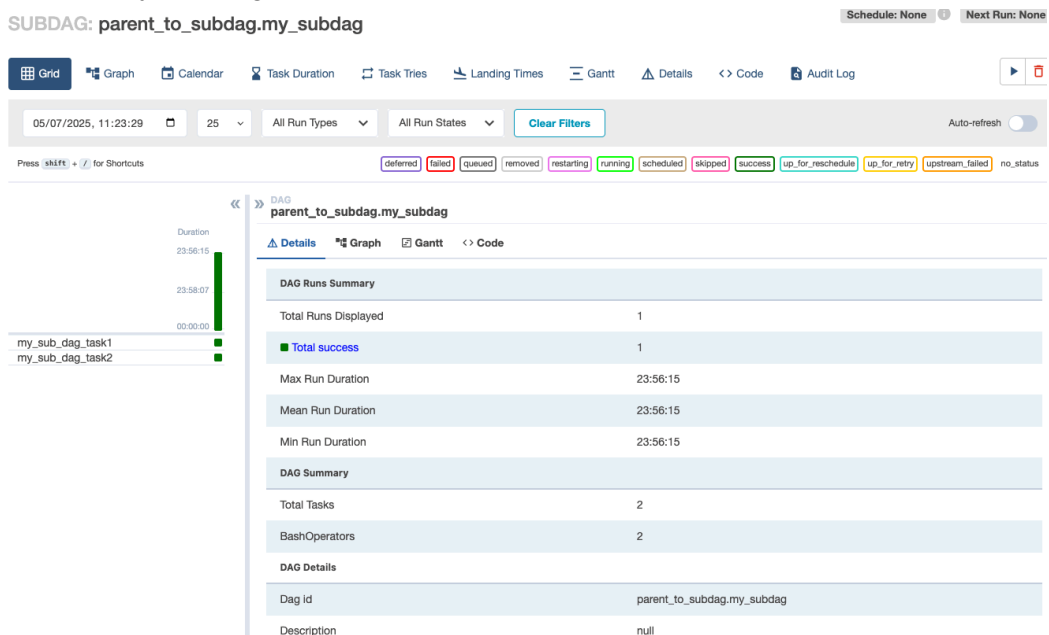


The image shows the Airflow web interface's sign-in page. At the top left is the Airflow logo, and at the top right is the time '11:17 UTC' and a 'Log In' link. The main content is a 'Sign In' box with the instruction 'Enter your login and password below:'. It contains two input fields: 'Username:' with 'airflow' entered, and 'Password:' with 'airflow' entered (masked with dots). A blue 'Sign in' button is at the bottom of the box.

Search for DAG parent_to_subdag in Airflow UI and trigger by clicking on the play button



Click on my_subdag node -> Zoom into SubDAG -> observe the two bash-tasks

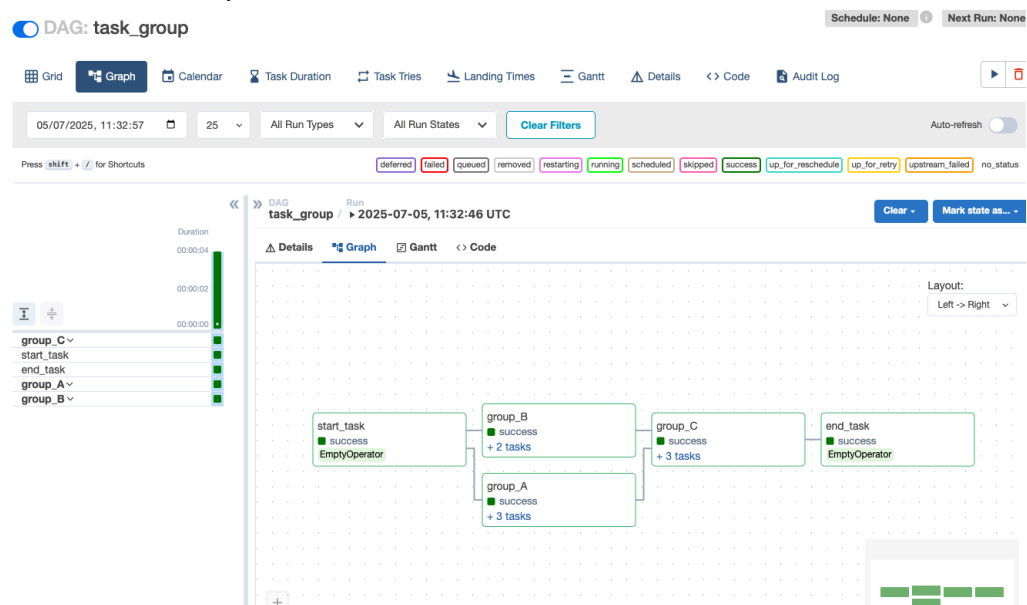


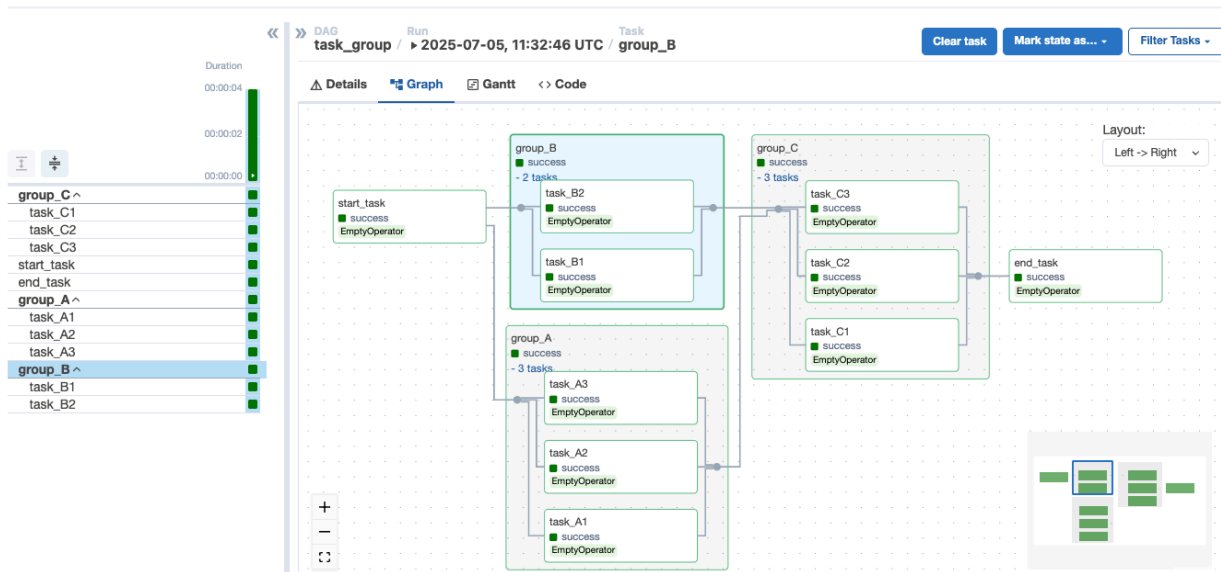
TaskGroup:

Create my_taskgroup.py file under dags/

```
my_taskgroup.py U X
tascientest-data-engineering > airflow > dags > my_taskgroup.py
1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.operators.dummy import DummyOperator
4 from airflow.utils.task_group import TaskGroup
5
6
7 with DAG(
8     dag_id="task_group",
9     schedule_interval=None,
10    tags=['tutorial', 'datascientest'],
11    default_args={
12        'owner': 'airflow',
13        'start_date': days_ago(0, 1)
14    }
15 ) as my_dag:
16     with TaskGroup("group_A") as group_A:
17         task_A1 = DummyOperator(task_id="task_A1")
18         task_A2 = DummyOperator(task_id="task_A2")
19         task_A3 = DummyOperator(task_id="task_A3")
20
21     with TaskGroup("group_B") as group_B:
22         task_B1 = DummyOperator(task_id="task_B1")
23         task_B2 = DummyOperator(task_id="task_B2")
24
25     with TaskGroup("group_C") as group_C:
26         task_C1 = DummyOperator(task_id="task_C1")
27         task_C2 = DummyOperator(task_id="task_C2")
28         task_C3 = DummyOperator(task_id="task_C3")
29
30     start_task = DummyOperator(task_id='start_task')
31     end_task = DummyOperator(task_id='end_task')
32
33     start_task >> [group_A, group_B]
34     [group_A, group_B] >> group_C >> end_task
```

Search for DAG task_group in Airflow UI and trigger by clicking on the play button. Graph view shows collapsible boxes for A, B, C.



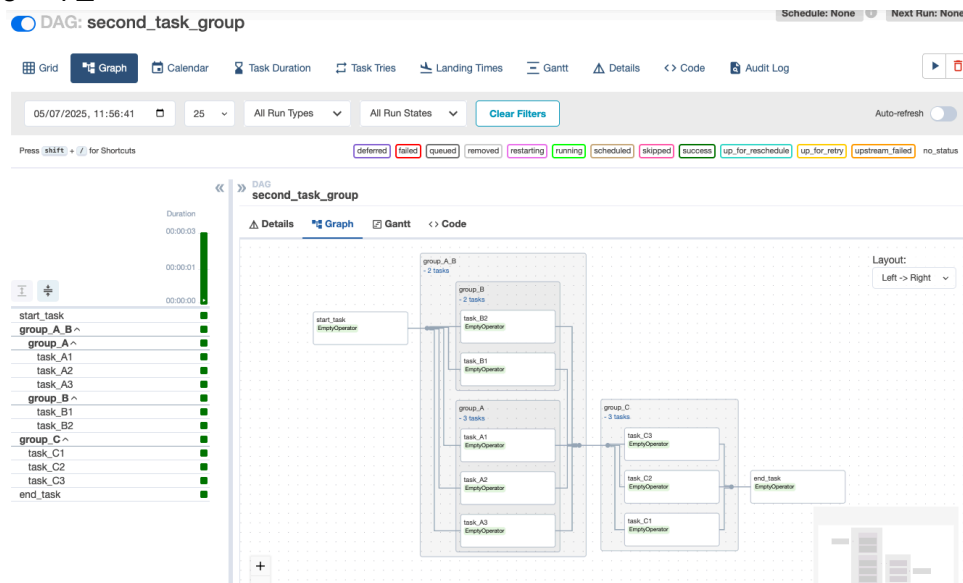


Create second_taskgroup.py file under dags/

```

second_taskgroup.py
1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.operators.dummy import DummyOperator
4 from airflow.utils.task_group import TaskGroup
5
6
7 with DAG(
8     dag_id="second_task_group",
9     schedule_interval=None,
10    tags=["tutorial", "datascientest"],
11    default_args={
12        "owner": "airflow",
13        "start_date": days_ago(0, 1)
14    }
15 ) as my_dag:
16     with TaskGroup("group_A_B") as group_A_B:
17         with TaskGroup("group_A") as group_A:
18             task_A1 = DummyOperator(task_id="task_A1")
19             task_A2 = DummyOperator(task_id="task_A2")
20             task_A3 = DummyOperator(task_id="task_A3")
21
22         with TaskGroup("group_B") as group_B:
23             task_B1 = DummyOperator(task_id="task_B1")
24             task_B2 = DummyOperator(task_id="task_B2")
25
26     with TaskGroup("group_C") as group_C:
27         task_C1 = DummyOperator(task_id="task_C1")
28         task_C2 = DummyOperator(task_id="task_C2")
29         task_C3 = DummyOperator(task_id="task_C3")
30
31     start_task = DummyOperator(task_id="start_task")
32     end_task = DummyOperator(task_id="end_task")
33
34     start_task >> group_A_B >> group_C >> end_task
  
```

Search for DAG `second_task_group` in Airflow UI and trigger by clicking on the play button. Graph view will show a single collapsible box `group_A_B` wrapping both `group_A` and `group_B`.

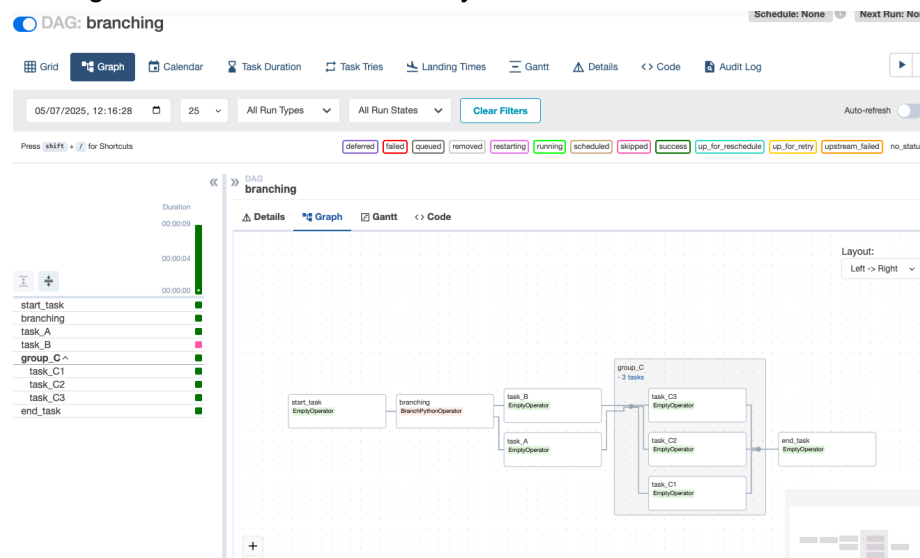


Branching:

Create `branching.py` file under `dags/`

```
branching.py U X
atascientest-data-engineering > airflow > dags > branching.py
1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.operators.dummy import DummyOperator
4 from airflow.utils.task_group import TaskGroup
5 from airflow.operators.python import BranchPythonOperator
6 import random
7
8
9 def decide_branch(condition):
10     if condition:
11         return "task_A"
12     else:
13         return "task_B"
14
15 with DAG(
16     dag_id="branching",
17     schedule_interval=None,
18     tags=["tutorial", "datascientest"],
19     default_args={
20         "owner": "airflow",
21         "start_date": days_ago(0, 1)
22     }
23 ) as my_dag:
24     start_task = DummyOperator(task_id="start_task")
25
26     branch_decider = BranchPythonOperator(
27         task_id="branching",
28         python_callable=decide_branch,
29         op_args={"condition": bool(random.getrandbits(1))}
30     )
31
32     task_A = DummyOperator(task_id="task_A")
33     task_B = DummyOperator(task_id="task_B")
34
35     with TaskGroup("group_C") as group_C:
36         task_C1 = DummyOperator(task_id="task_C1", trigger_rule="all_done")
37         task_C2 = DummyOperator(task_id="task_C2", trigger_rule="all_done")
38         task_C3 = DummyOperator(task_id="task_C3", trigger_rule="all_done")
39
40     end_task = DummyOperator(task_id="end_task", trigger_rule="all_done")
41
42     start_task >> branch_decider
43     branch_decider >> [task_A, task_B]
44     [task_A, task_B] >> group_C >> end_task
```

Search for DAG branching in Airflow UI and trigger by clicking on the play button. When running the DAG, observe that only task A or task B has been executed



Xcoms:

Create simple_xcom_dag.py file under dags/

```
simple_xcom_dag.py
1 from airflow import DAG
2 from airflow.operators.python_operator import PythonOperator
3 from airflow.utils.dates import days_ago
4 import random
5
6
7 def function_with_return():
8     return random.uniform(a=0, b=1)
9
10 with DAG(
11     dag_id="simple_xcom_dag",
12     schedule_interval=None,
13     tags=["tutorial", "datascientest"],
14     start_date=days_ago(0)
15 ) as dag:
16     my_task = PythonOperator(
17         task_id="python_task",
18         python_callable=function_with_return
19     )
```

Go to Admin -> XComs to observe the value returned by the task. The value is unnamed and therefore difficult to reuse.

Key	Value
return_value	0.31149038381181726

Modify the simple_xcom_dag.py file

```
simple_xcom_dag.py U X
tascientest-data-engineering > airflow > dags > simple_xcom_dag.py
1  from airflow import DAG
2  from airflow.operators.python_operator import PythonOperator
3  from airflow.utils.dates import days_ago
4  import random
5
6
7  def function_with_return(task_instance):
8      task_instance.xcom_push(
9          key = "my_xcom_value",
10         value = random.uniform(a=0, b=1)
11     )
12
13 with DAG(
14     dag_id = "simple_xcom_dag",
15     schedule_interval = None,
16     tags = ["tutorial", "datascientest"],
17     start_date = days_ago(0)
18 ) as dag:
19     my_task = PythonOperator(
20         task_id = "python_task",
21         python_callable = function_with_return
22     )
```

Trigger the DAG again in Airflow UI

DAG: simple_xcom_dag Schedule: False

Grid Graph Calendar Task Duration Task Tries Landing Times Gantt Details <> Code Audit Log

Task Instance: **python_task** at 2025-07-05, 12:33:44

Task Instance Details <> Rendered Template Log **XCom**

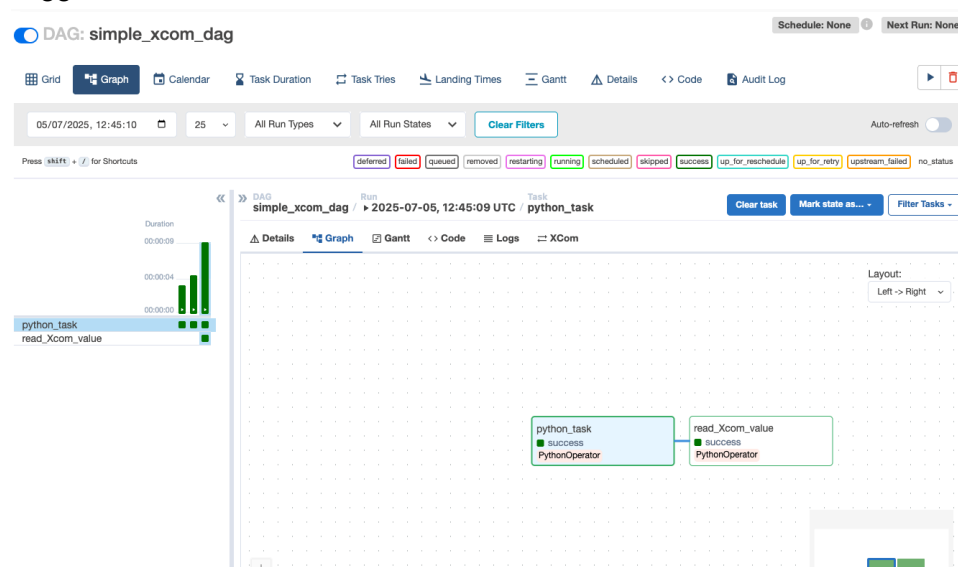
XCom

Key	Value
my_xcom_value	0.5057231961846574

Modify `simple_xcom_dag.py` again to add a second task to the previous DAG displaying contents of the variable produced in the first task.

```
simple_xcom_dag.py U X
tascientest-data-engineering > airflow > dags > simple_xcom_dag.py
1 from airflow import DAG
2 from airflow.operators.python_operator import PythonOperator
3 from airflow.utils.dates import days_ago
4 import random
5
6
7 def function_with_return(task_instance):
8     task_instance.xcom_push(
9         key = "my_xcom_value",
10        value = random.uniform(a=0, b=1)
11    )
12
13 def read_data_from_xcom(task_instance):
14     print(
15         task_instance.xcom_pull(
16             key = "my_xcom_value",
17             task_ids = ["python_task"]
18         )
19     )
20
21 with DAG(
22     dag_id = "simple_xcom_dag",
23     schedule_interval = None,
24     tags = ["tutorial", "datascientest"],
25     start_date = days_ago(0)
26 ) as dag:
27     my_task = PythonOperator(
28         task_id = "python_task",
29         python_callable = function_with_return
30     )
31
32     my_task2 = PythonOperator(
33         task_id = "read_xcom_value",
34         python_callable = read_data_from_xcom
35     )
36
37     my_task >> my_task2
```

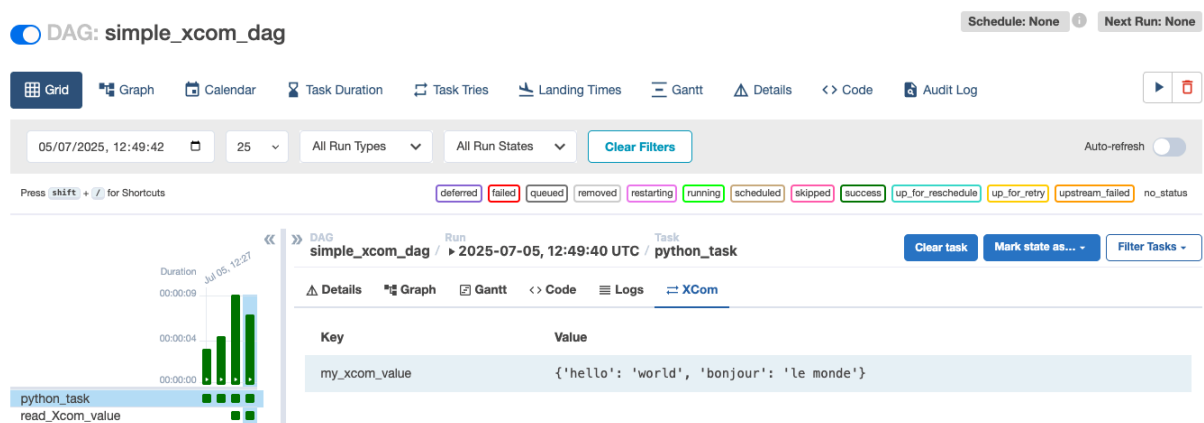
Trigger the DAG in Airflow UI



Modify `simple_xcom_dag.py` again to transform the previous DAG so that it registers an XCom whose value is a dictionary with several keys.

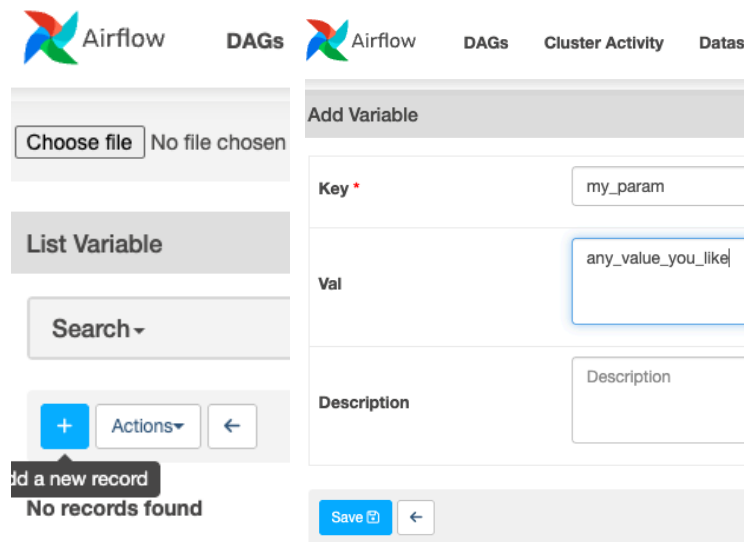
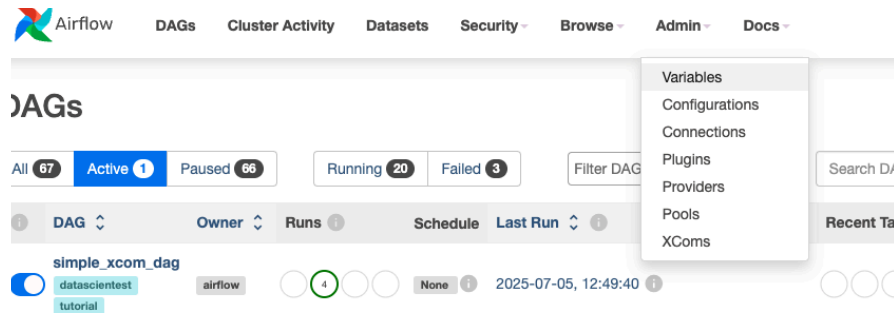
```
simple_xcom_dag.py U X
tascientest-data-engineering > airflow > dags > simple_xcom_dag.py
1  from airflow import DAG
2  from airflow.operators.python_operator import PythonOperator
3  from airflow.utils.dates import days_ago
4  import random
5
6
7  def function_with_return(task_instance):
8      task_instance.xcom_push(
9          key = "my_xcom_value",
10         value = {
11             "hello": "world",
12             "bonjour": "le monde"
13         }
14     )
15
16 def read_data_from_xcom(task_instance):
17     print(
18         task_instance.xcom_pull(
19             key = "my_xcom_value",
20             task_ids = ["python_task"]
21         )
22     )
23
24 with DAG(
25     dag_id = "simple_xcom_dag",
26     schedule_interval = None,
27     tags = ["tutorial", "datascientest"],
28     start_date = days_ago(0)
29 ) as dag:
30     my_task = PythonOperator(
31         task_id = "python_task",
32         python_callable = function_with_return
33     )
34
35     my_task2 = PythonOperator(
36         task_id = "read_Xcom_value",
37         python_callable = read_data_from_xcom
38     )
39
40     my_task >> my_task2
```

Trigger the DAG in Airflow UI



Variables:

Navigate to Airflow UI -> Admin -> Variables -> + -> Key = my_param, Val = any_value_you_like -> Save



Create var_read_dag.py under dags/

```
var_read_dag.py U X
tascientest-data-engineering > airflow > dags > var_read_dag.py
1 from airflow import DAG
2 from airflow.models import Variable
3 from airflow.operators.python import PythonOperator
4 from airflow.utils.dates import days_ago
5
6
7 def print_var():
8     print("my_param =", Variable.get("my_param"))
9
10 with DAG(
11     dag_id = "var_reader_dag",
12     schedule_interval = None,
13     tags = ["tutorial", "datascientest"],
14     start_date = days_ago(0)
15 ) as dag:
16     read = PythonOperator(
17         task_id = "print_variable",
18         python_callable = print_var
19     )
```

Trigger the DAG var_reader_dag in Airflow UI, check the logs, observe print of my_param value

The screenshot shows the Airflow web interface. At the top, the DAG is named 'var_reader_dag'. Below the DAG name, there are tabs for 'Grid', 'Graph', 'Calendar', 'Task Duration', 'Task Tries', 'Landing Times', 'Gantt', 'Details', 'Code', and 'Audit Log'. The 'Details' tab is selected. On the left, there is a 'print_variable' task listed. The main panel shows the task 'var_reader_dag / 2025-07-05, 13:02:50 UTC / print_variable'. The task is in a 'running' state. The logs show the execution of the task, including the print statement for my_param.

```
35a3cfe351c7
*** Found local files:
*** * /opt/airflow/logs/dag_id=var_reader_dag/run_id=manual_2025-07-05T13:02:50.374792+00:00/task_id=print_variable/attempt=1.log
[2025-07-05, 13:02:54 UTC] {taskinstance.py:1956} INFO - Dependencies all met for dep_context=non-requeueable deps ti=<TaskInstance: var_reader_dag.print_variable>
[2025-07-05, 13:02:54 UTC] {taskinstance.py:1956} INFO - Dependencies all met for dep_context=requeueable deps ti=<TaskInstance: var_reader_dag.print_variable>
[2025-07-05, 13:02:54 UTC] {taskinstance.py:2170} INFO - Starting attempt 1 of 1
[2025-07-05, 13:02:54 UTC] {taskinstance.py:2191} INFO - Executing <Task(PythonOperator): print_variable> on 2025-07-05 13:02:50.374792+00:00
[2025-07-05, 13:02:54 UTC] {standard_task_runner.py:60} INFO - Started process 3686 to run task
[2025-07-05, 13:02:54 UTC] {standard_task_runner.py:87} INFO - Running: ['***', 'tasks', 'run', 'var_reader_dag', 'print_variable', 'manual_2025-07-05T13:02:50.374792+00:00']
[2025-07-05, 13:02:54 UTC] {standard_task_runner.py:88} INFO - Job 2031: Subtask print_variable
[2025-07-05, 13:02:54 UTC] {warnings.py:109} WARNING - /home/***/.local/lib/python3.8/site-packages/***/settings.py:194: DeprecationWarning: The sqlalchemy_alembic_conf = conf.get('database', "SQLALCHEMY_CONF")
[2025-07-05, 13:02:54 UTC] {task_command.py:423} INFO - Running <TaskInstance: var_reader_dag.print_variable manual_2025-07-05T13:02:50.374792+00:00 [run]
[2025-07-05, 13:02:55 UTC] {taskinstance.py:2480} INFO - Exporting env vars: AIRFLOW_CTX_DAG_OWNER='***' AIRFLOW_CTX_DAG_ID='var_reader_dag' AIRFLOW_CTX_TASK_ID='print_variable'
[2025-07-05, 13:02:55 UTC] {logging_mixin.py:188} INFO - my_param = any_value_you_like
[2025-07-05, 13:02:55 UTC] {python.py:201} INFO - Done. Returned value was: None
[2025-07-05, 13:02:55 UTC] {taskinstance.py:1130} INFO - Marking task as SUCCESS. dag_id=var_reader_dag, task_id=print_variable, execution_date=20250705T13:02:50.374792+00:00
[2025-07-05, 13:02:55 UTC] {local_task_job_runner.py:234} INFO - Task exited with return code 0
[2025-07-05, 13:02:55 UTC] {taskinstance.py:3280} INFO - 0 downstream tasks scheduled from follow-on schedule check
```

DAG Decorators:

Create decorator_dag.py under dags/

```
decorator_dag.py U X
atascientest-data-engineering > airflow > dags > decorator_dag.py
1 from airflow import DAG
2 from airflow.decorators import dag, task
3 from airflow.utils.dates import days_ago
4 from airflow.operators.python import get_current_context
5 import random
6
7
8 @task
9 def function_with_return():
10     return random.uniform(a=0, b=1)
11
12 @task
13 def function_with_return_and_push():
14     task_instance = get_current_context()["task_instance"]
15     value = random.uniform(a=0, b=1)
16     task_instance.xcom_push(key = "my_xcom_value", value = value)
17     return value
18
19 @task
20 def read_data_from_xcom(my_xcom_value):
21     print(my_xcom_value)
22
23 @dag(
24     dag_id = "decorator_xcom_dag",
25     tags = ["tutorial", "datascientest"],
26     schedule_interval = None,
27     start_date = days_ago(0)
28 )
29 def my_dag():
30     my_task1 = function_with_return_and_push()
31     my_task2 = read_data_from_xcom(my_task1)
32
33 my_dag = my_dag()
```

Trigger the DAG decorator_xcom_dag in Airflow UI Trigger. Graph view will the two tasks execute in order, passing the value via XCom under the hood.

